

## USER MANUAL

# ESP32 Temperature, Humidity, and Gas Monitoring System

Operation, setup, safety, and maintenance guide

|               |                                                  |
|---------------|--------------------------------------------------|
| Course        | Architecture and Organization, 2nd Semester 2025 |
| Instructor    | Christopher Aris Alviola                         |
| Project Owner | Jose Emmanuel "Joem" Ridpan                      |
| Device        | ESP32 with DHT11 and MQ135 sensors               |

**Important Safety Notice**

This system is a school project prototype. Use a proper 5V USB power source only. Do not connect unprotected 5V analog outputs directly to ESP32 input pins.

Prepared as a reference manual for installation, operation, troubleshooting, and support.

**Manual Scope**

This document covers introduction, hardware and software setup, usage, troubleshooting, maintenance, and support information for the ESP32 monitoring system.

## ESP32 Temperature, Humidity, and Gas Monitoring System

**Course:** Architecture and Organization, 2nd Semester 2025

**Prepared for:** Christopher Aris Alviola

**Project Owner:** Jose Emmanuel "Joem" Ridpan

### 1. Introduction

This project is an ESP32-based environmental monitoring system designed to measure **temperature**, **humidity**, and **gas levels** using connected sensors. The system reads data from a **DHT11** sensor and an **MQ135** gas sensor, then displays the readings through a **local web dashboard** and sends telemetry to **Arduino Cloud**. It also stores session data using **Google Sheets with Google Apps Script** as the backend.

#### Purpose of the project

- Monitor surrounding environmental conditions in real time
- Provide a simple local dashboard accessible through a browser
- Store sensor readings for later review
- Demonstrate practical ESP32 integration with sensors, web interface, and cloud services

#### Benefits of the monitoring system

- Real-time visibility of temperature, humidity, and gas trends
- Browser-based local dashboard with live graphs
- Session-based data logging for record keeping
- Cloud monitoring through Arduino Cloud
- CSV export for saved data analysis

### 2. Getting Started

Before using the system, the user should understand that this project combines **hardware**, **firmware**, **local networking**, and **cloud logging**.

#### Prerequisites

- ESP32 development board
- DHT11 temperature and humidity sensor
- MQ135 gas sensor module
- Jumper wires or properly soldered hookup wires
- USB Type-C cable for the ESP32
- Laptop or PC with Arduino IDE installed
- WiFi connection with internet access
- Arduino Cloud account
- Google account for Google Sheets and Apps Script
- Enclosure with ventilation holes

#### Important notes before starting

- The ESP32 uses **3.3V logic**, so sensor wiring must be done carefully.
- The **MQ135 analog output can be unsafe for ESP32 input if powered at 5V without protection**.
- The enclosure should not be fully sealed because sensors need airflow.
- This system is intended for **indoor or controlled use**, not outdoor deployment.

## 3. Hardware Setup

### Main hardware components

- ESP32 board
- DHT11 sensor
- MQ135 gas sensor
- USB Type-C cable
- Enclosure with side or bottom ventilation holes

### Recommended wiring

#### DHT11

- **VCC -> 3.3V**
- **GND -> GND**
- **DATA -> GPIO 4**

If using a bare DHT11 and not a module, add a **10k pull-up resistor** from DATA to VCC.

#### MQ135

- **GND -> GND**
- **AO -> GPIO 32**
- **DO -> Not used**

### MQ135 power options

#### Safer option

- **VCC -> 3.3V**
- **AO -> GPIO 32 directly**

#### Alternative option

- **VCC -> 5V**
- Use a **voltage divider** between AO and GPIO 32

Example divider:

- **20k resistor** from AO to GPIO32
- **10k resistor** from GPIO32 to GND

### Physical setup steps

1. Place the ESP32 securely inside the enclosure.
2. Position the DHT11 and MQ135 so they are exposed to air through the ventilation holes.
3. Connect all wires according to the recommended wiring above.
4. Inspect all joints and wiring before powering the system.
5. Make sure there are **no solder bridges or shorts**.
6. Connect the ESP32 using a **USB Type-C cable**.

### Powering the device

Accepted power sources:

- Laptop USB port
- Power bank
- Proper 5V USB wall adapter

Not recommended:

- Directly improvising power from an outlet
- Unregulated power sources

Use only a proper USB power source.

## 4. Software Setup

### A. Install required software

Install the following:

- **Arduino IDE**
- **ESP32 board package** by Espressif
- Required libraries:
- ArduinoIoTCloud
- Arduino\_ConnectionHandler
- DHT sensor library
- Adafruit Unified Sensor

### B. Open the project files

Main sketch location:

- D:\repos\GasSensor-ESP32\ESP32\_Cloud\ESP32\_Cloud\ESP32\_Cloud.ino

Related files:

- D:\repos\GasSensor-ESP32\ESP32\_Cloud\ESP32\_Cloud\thingProperties.h
- D:\repos\GasSensor-ESP32\ESP32\_Cloud\ESP32\_Cloud\arduino\_secrets.h
- D:\repos\GasSensor-ESP32\GoogleSheets-Backend\Code.gs

### C. Configure Arduino Cloud

1. Go to <https://cloud.arduino.cc>
2. Create or sign in to an Arduino account.
3. Create a new **Thing**.
4. Add a **3rd party ESP32 device**.
5. Create the following cloud variables:
  - cloudTemperature as float, Read Only
  - cloudHumidity as float, Read Only
  - cloudGas as float or numeric equivalent, Read Only
6. Place the generated credentials in the project files if needed.
7. Upload the sketch through Arduino IDE.

### D. Configure WiFi and secrets

In `arduino_secrets.h`, place:

- WiFi SSID
- WiFi password
- Arduino Cloud device key

- Google Apps Script backend URL

## E. Configure Google Sheets backend

1. Create a new Google Sheet.
2. Open **Extensions > Apps Script**.
3. Replace the default script with the contents of:
  - D:\repos\GasSensor-ESP32\GoogleSheets-Backend\Code.gs
4. Deploy as **Web App**.
5. Set:
  - **Execute as:** Me
  - **Who has access:** Anyone
6. Copy the deployment URL.
7. Put that URL into `arduino_secrets.h` as the backend URL.
8. Redeploy the Apps Script whenever the backend code is updated.

## F. About Blynk

The current system architecture uses **Arduino Cloud** and **Google Sheets + Apps Script**.

**Blynk is not required in the current implementation.**

## G. Uploading the code

1. Connect the ESP32 to the computer.
2. Open Arduino IDE.
3. Select the correct ESP32 board.
4. Select the correct COM port.
5. Upload the sketch.
6. Open Serial Monitor at **115200 baud**.
7. Wait for the ESP32 to connect to WiFi and print its local IP address.

# 5. Usage Instructions

## Accessing live data

After upload and successful WiFi connection:

1. Connect your phone or laptop to the **same WiFi network** as the ESP32.
2. Open the IP address shown in the Serial Monitor.
3. The local webpage will display:
  - Temperature
  - Humidity
  - Gas level
  - Live graphs
  - Session information

## Available local dashboard routes

- / -> main dashboard
- /data -> current sensor data
- /history -> recent local history
- /download -> downloadable data output

## Cloud usage

Arduino Cloud receives:

- Temperature
- Humidity
- Gas readings

This allows cloud-side monitoring in addition to the local webpage.

### Session logging

- Each power-on run is treated as one **session**.
- Readings are sent to the Google Sheets backend.
- Saved sessions can be reviewed from the web interface.
- Data can also be exported for analysis.

### Important usage note about gas values

The gas reading is currently a **raw ADC value**, not a calibrated PPM value.

This is acceptable for prototype and academic demonstration purposes, but it should not be treated as a certified gas measurement.

## 6. Troubleshooting

### 1. ESP32 does not power on

- Check the USB cable.
- Try another USB port.
- Use a known working laptop USB port or power bank.
- Avoid suspicious power adapters.

### 2. Code does not upload

- Check that the correct ESP32 board is selected.
- Check the correct COM port.
- Reconnect the board.
- Install the ESP32 board package if missing.

### 3. No readings from DHT11

- Check VCC, GND, and DATA wiring.
- Confirm DATA is connected to **GPIO 4**.
- If using a bare sensor, make sure there is a **10k pull-up resistor**.
- Check for loose wires or cold solder joints.

### 4. MQ135 readings seem wrong or unstable

- Check that AO is connected to **GPIO 32**.
- Make sure the module is powered correctly.
- If using 5V on MQ135, verify the voltage divider is present.
- Inspect for poor soldering or unstable power.

### 5. ESP32 connects to WiFi but local webpage does not open

- Make sure the viewing device is on the **same network**.
- Recheck the IP address in Serial Monitor.
- Refresh the page.
- Try accessing from a laptop if a phone hotspot setup fails.

## 6. Arduino Cloud does not update

- Recheck the Thing ID, device credentials, and cloud variable names.
- Verify WiFi internet connection.
- Confirm all required libraries are installed.

## 7. Google Sheets backend does not save data

- Check that the Apps Script web app is deployed.
- Confirm the URL in `arduino_secrets.h` is correct.
- Redeploy the Apps Script after backend changes.
- Test the web app URL manually in a browser.

## 8. Device resets or behaves strangely

- Check for shorts, solder bridges, and weak connections.
- Inspect power and GND lines carefully.
- Verify the MQ135 analog signal is not exceeding safe ESP32 voltage.

# 7. Maintenance

To keep the system functioning properly, perform the following maintenance regularly:

### Hardware maintenance

- Inspect wiring and solder joints for looseness or corrosion.
- Keep sensor surfaces free from dust.
- Make sure the enclosure ventilation holes are not blocked.
- Do not expose the system to rain or outdoor moisture.
- Avoid trapping heat inside the enclosure.

### Software maintenance

- Back up the Arduino sketch and backend files.
- Redeploy Google Apps Script if backend logic is changed.
- Recheck Arduino Cloud settings if cloud sync stops working.

### Safety maintenance

- Inspect power wiring before each major use.
- Do not power the system if there are visible shorts or damaged wires.
- Replace damaged jumper wires or connectors immediately.

# 8. Warranty and Support

This project is an academic prototype for educational use. No formal commercial warranty is provided.

### Support contact

For technical questions, setup concerns, or project support, contact:

- **Website:** [joemidpan.com](http://joemidpan.com)
- **Email:** [jridpan1225@gmail.com](mailto:jridpan1225@gmail.com)

## Summary

This ESP32 monitoring system provides:

- Real-time temperature, humidity, and gas monitoring
- Local browser dashboard access
- Arduino Cloud telemetry
- Google Sheets session logging
- Exportable session data for analysis

It is intended as a functional academic project prototype with practical monitoring features and expandable architecture.